



基于哈夫曼编码的文件压缩软件设计汇报

小组成员:

时间:



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

目录

CONTENTS

1

基本情况

2

设计与实现

3

测试与结论

4

总结与思考



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

1

基本情况

题目内容

设计并实现一款基于哈夫曼编码的文件压缩软件，实现文件的无损压缩与解压。通过分析文件中的字符频率，建立字符集频率表并由此构建哈夫曼树对源文件进行编码，将源文件压缩为更小的二进制文件；同时支持将压缩后的文件解压缩为原始文件。



题目要求及约束条件

我们设计的压缩软件可以从源文件中**读取数据**，并统计各字符出现的频率**生成字符集频率表**。基于字符集频率表**构建哈夫曼树**对源文件**进行编码**，生成.huff 格式文件。利用构建的哈夫曼树对.huff 格式文件**进行解压**，还原为解压文件。设计的压缩软件支持输入源文件为任意格式，并且解压文件与压缩文件相比无失真。压缩文件需要自定义文件头存储关键信息（存储频率表、编码位数等元数据），解压时需读取文件头重建哈夫曼树。

1



2



3



4



5



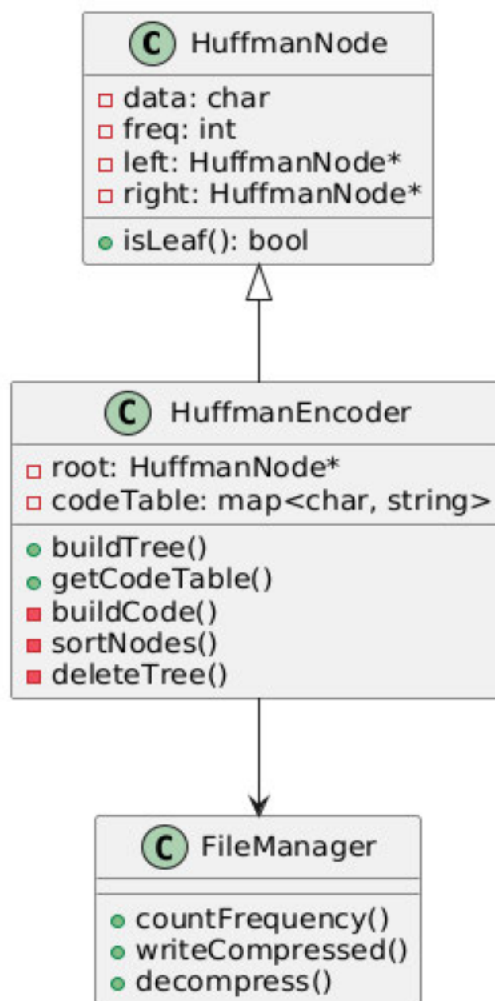


北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

2

设计与实现

在这个信息高速发展的时代，我们需要存储并传输大量的文件。在占用存储空间较大的文件中通常有**大量冗余**，因而我们可以对其进行编码实现**文件压缩**。在我们设计的压缩软件中，我们通过**哈夫曼编码**实现对**任意格式源文件**的**压缩和解压**。在压缩过程中首先从源文件中**读取数据**并建立**字符集频率表**，利用建立的字符集频率表完成**哈夫曼树的构建**并对源文件**进行编码**，最后将字符种类数和编码表写入文件头，压缩数据按照**每八位码符号一字节**写入.huff文件正文实现对源文件的压缩。在解压过程中，首先通过读取文件头**重建解码映射表**，再根据前缀码的特性通过持续匹配编码完成对压缩文件的解压。**我们设计的压缩软件还可以对源文件为空，指针异常等情况进行处理，更能满足实际情况的需求。**



哈夫曼树节点
(HuffmanNode)类:
表示哈夫曼树的单个
节点, 存储该节点对
应的字符数据、出现
频率和指向其左右孩
子节点的指针, 并提
供了叶子结点的判断
方法

```
// ===== 哈夫曼节点结构 =====  
class HuffmanNode {  
private:  
    char data;        // 存储的字符 (0-255)  
    int freq;         // 字符出现次数  
    HuffmanNode* left; // 左子树指针  
    HuffmanNode* right; // 右子树指针  
  
public:  
    friend class HuffmanEncoder; // 声明友元类以便访问私有成员  
  
    // 构造函数  
    HuffmanNode(char d, int f)  
    : data(d), freq(f), left(nullptr), right(nullptr) {}  
  
    // 判断是否是叶子节点 (没有子节点)  
    bool isLeaf() const {  
        return !left && !right;  
    }  
};
```

哈夫曼编码器
(HuffmanEncoder)类:
构建哈夫曼树并对每个
叶子节点的字符数据进
行哈夫曼编码。存储构
造的哈夫曼树的根节点
以及编码表, 提供了获
取源文件的编码表, 构
建哈夫曼树并进行哈夫
曼编码的方法。

```
// ===== 哈夫曼编码器 =====
class HuffmanEncoder {
private:
    HuffmanNode* root; // 哈夫曼树的根节点
    map<char, string> codeTable; // 字符到二进制编码的映射表

    // 递归生成字符编码 (核心方法)
    void buildCode(HuffmanNode* node, string code) {
        if (!node) return; // 递归终止条件

        // 叶子节点存储编码
        if (node->isLeaf()) {
            codeTable[node->data] = code;
            return;
        }

        // 左子树编码追加0, 右子树追加1
        buildCode(node->left, code + "0");
        buildCode(node->right, code + "1");
    }

    // 手动实现节点排序 (冒泡排序)
    void sortNodes(HuffmanNode** nodes, int count) {
        // 每次把最大的元素冒泡到末尾
        for (int i = 0; i < count-1; ++i) {
            for (int j = 0; j < count-i-1; ++j) {
                // 比较频率大小
                if (nodes[j]->freq > nodes[j+1]->freq) {
                    swap(nodes[j], nodes[j+1]); // 交换节点位置
                }
            }
        }
    }

    // 递归删除树节点 (用于析构函数)
    void deleteTree(HuffmanNode* node) {
        if (node) {
            deleteTree(node->left);
            deleteTree(node->right);
            delete node; // 后序遍历删除
        }
    }

public:
    // 构造函数
    HuffmanEncoder() : root(nullptr) {}

    // 析构函数 (防止内存泄漏)
    ~HuffmanEncoder() {
        deleteTree(root);
    }
};
```

```
// 构建哈夫曼树 (核心方法)
void buildTree(const map<char, int>& freqMap) {
    const int MAX_NODES = 512; // 最大节点数 (256个字符需要511个节点)
    HuffmanNode* nodes[MAX_NODES] = {nullptr}; // 初始化指针数组
    int nodeCount = 0;

    // 处理空文件特殊情况
    if (freqMap.empty()) {
        root = nullptr;
        codeTable.clear();
        return;
    }

    // 第一步: 创建所有叶节点
    for (map<char, int>::const_iterator it = freqMap.begin();
         it != freqMap.end(); ++it) {
        nodes[nodeCount++] = new HuffmanNode(it->first, it->second);
    }

    // 第二步: 构建哈夫曼树
    while (nodeCount > 1) {
        // 按频率升序排序
        sortNodes(nodes, nodeCount);

        // 取出两个最小节点
        HuffmanNode* left = nodes[0];
        HuffmanNode* right = nodes[1];

        // 创建新父节点 (字符设为'\0')
        HuffmanNode* parent = new HuffmanNode('\0', left->freq + right->freq);
        parent->left = left;
        parent->right = right;

        // 更新节点数组
        nodes[0] = parent; // 父节点占据第一个位置
        for (int i = 1; i < nodeCount-1; ++i) {
            nodes[i] = nodes[i+1]; // 后续节点前移
        }
        nodeCount--; // 总节点数减1
    }

    root = nodes[0]; // 最后的节点即为根节点
    buildCode(root, ""); // 生成编码表

    // 获取编码表 (供文件写入使用)
    const map<char, string>& getCodeTable() const {
        return codeTable;
    }
};
```

文件管理
(FileManager)类:
封装了统计字符频率、写入压缩文件、解压缩文件等函数,使代码的逻辑结构更加清晰。

```
// ===== 文件管理类 =====
class FileManager {
public:
    // 统计字符频率 (返回频率表)
    static map<char, int> countFrequency(const string& filename) {
        ifstream fin(filename, ios::binary);
        if (!fin) {
            cerr << "错误, 无法打开文件 " << filename << endl;
            exit(1);
        }

        map<char, int> freqMap;
        char ch;
        while (fin.get(ch)) { // 逐个字符读取
            freqMap[ch]++; // 统计频率
        }
        fin.close();
        return freqMap;
    }

    // 写入压缩文件 (核心方法)
    static void writeCompressed(const string& inputFile,
                               const string& outputFile,
                               const map<char, string>& codeTable) {
        ifstream fin(inputFile, ios::binary);
        ofstream fout(outputFile, ios::binary);

        // 写入字符种类数 (4字节)
        int charCount = codeTable.size();
        fout.write(reinterpret_cast<const char*>(&charCount), sizeof(int));

        // 空文件特殊处理
        if (charCount == 0) {
            fin.close();
            fout.close();
            return;
        }

        // 写入编码表 (字符+编码长度+编码内容)
        for (map<char, string>::const_iterator it = codeTable.begin();
             it != codeTable.end(); ++it) {
            fout.put(it->first); // 写入字符 (1字节)
            unsigned char len = it->second.length();
            fout.put(len); // 编码长度 (1字节)
            fout.write(it->second.c_str(), len); // 编码内容
        }

        // 压缩数据
        unsigned char buffer = 0; // 位缓冲区
        int bitCount = 0; // 当前缓冲位数
        char ch;
```

```
while (fin.get(ch)) {
    const string& code = codeTable.at(ch);
    for (size_t i = 0; i < code.size(); ++i) {
        buffer = (buffer << 1) | (code[i] - '0'); // 左移并添加位
        if (++bitCount == 8) { // 缓冲区满
            fout.put(buffer);
            buffer = bitCount = 0;
        }
    }

    // 处理剩余位 (不足8位时补0)
    if (bitCount > 0) {
        buffer <<= (8 - bitCount); // 左移补零
        fout.put(buffer);
    }

    fin.close();
    fout.close();
}

// 解压文件 (核心方法)
static void decompress(const string& inputFile, const string& outputFile) {
    ifstream fin(inputFile, ios::binary);
    ofstream fout(outputFile, ios::binary);

    // 读取字符种类数
    int charCount;
    fin.read(reinterpret_cast<char*>(&charCount), sizeof(int));

    // 空文件处理
    if (charCount == 0) {
        fin.close();
        fout.close();
        return;
    }

    // 重建解码表 (编码到字符的映射)
    map<string, char> decodeMap;
    for (int i = 0; i < charCount; ++i) {
        char data;
        unsigned char codeLen;
        fin.get(data);
        fin.read(reinterpret_cast<char*>(&codeLen), 1);

        string code;
        code.resize(codeLen);
        fin.read(&code[0], codeLen); // 读取编码内容

        decodeMap[code] = data; // 建立映射关系
    }
```

```
// 解码数据
string bitStr; // 累积的二进制位字符串
char byte;
while (fin.get(byte)) {
    // 将字节转为8位二进制字符串
    for (int i = 7; i >= 0; --i) {
        bitStr += (byte & (1 << i)) ? '1' : '0';
    }

    // 持续匹配编码
    bool found;
    do {
        found = false;
        for (map<string, char>::const_iterator it = decodeMap.begin();
             it != decodeMap.end(); ++it) {
            if (bitStr.substr(0, it->first.size()) == it->first) {
                fout.put(it->second); // 写入解码字符
                bitStr = bitStr.substr(it->first.size()); // 移除已解码部分
                found = true;
                break;
            }
        }
    } while (found && !bitStr.empty());

    fin.close();
    fout.close();
}
```

数据结构

哈夫曼树

程序通过构建的字符集频率表建立哈夫曼树。哈夫曼树通过每个节点的左右孩子指针构成二叉树，并且每个叶子结点存储源文件中的字符数据及其出现频率。哈夫曼树是带权路径长度最小的二叉树，利用构建的哈夫曼树我们可以对源文件进行哈夫曼编码。

map容器

map是STL（中文标准模板库）的一个关联容器，可以将任何基本类型映射到任何基本类型。在本程序中通过map容器构建了编码表、频率表和解码表，分别实现了源文件字符到哈夫曼编码字符串、源文件字符到出现频率以及哈夫曼编码字符串到源文件字符的映射。

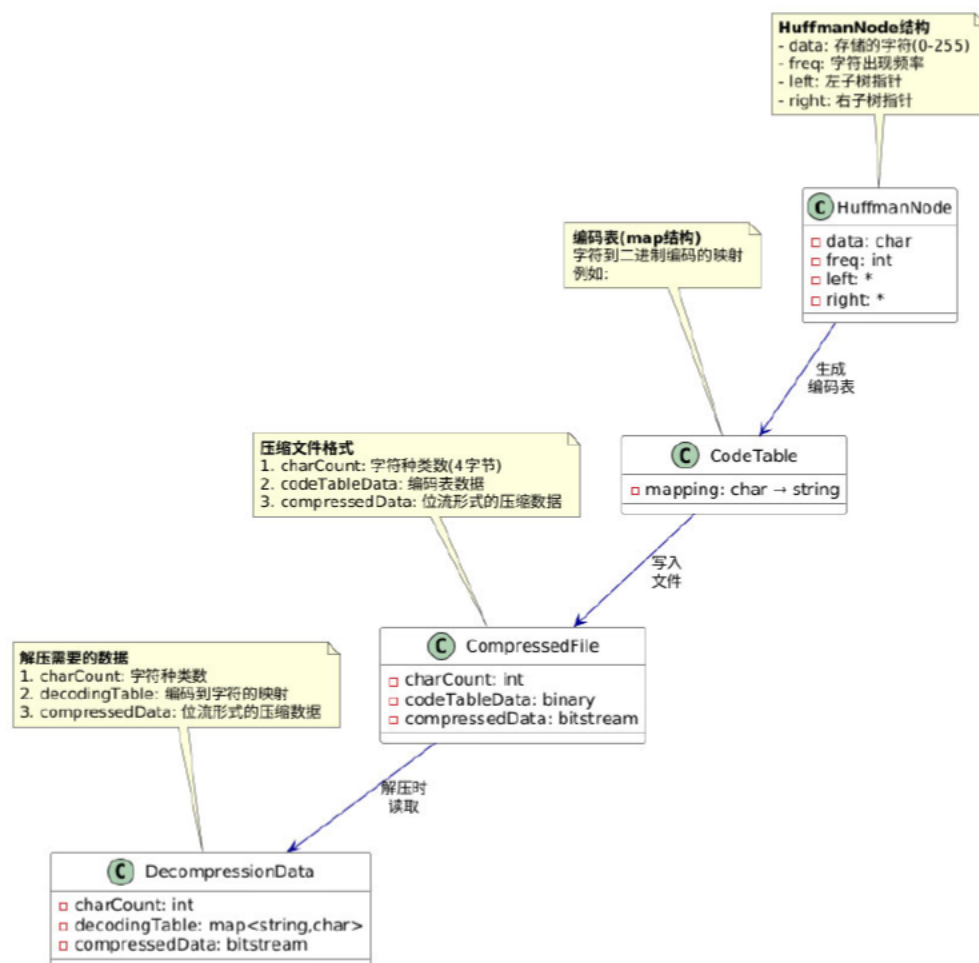
节点指针数组

在我们设计的压缩软件中，我们处理的源文件不同字符数一般不超过256个，因此构建的哈夫曼树需要511个节点。我们使用指针数组来存放指向这些节点的指针，使用这种顺序存储结构也有利于接下来哈夫曼树的构建。

德以明理 学以精工

优点

- ① 由哈夫曼树的结构可以直接使用递归的方法构造最优前缀码，并且采用二叉链表的存储方式使内存利用更高效。
- ② map容器按键值对的方式建立映射关系，使代码逻辑更加清晰；并且键从小到大的顺序自动排序，有利于程序的调试和测试。map容器无需预先分配内存，只存储实际出现的字符，有效提升内存利用率，并且时间复杂度为 $O(\log n)$ ，查询和更新高效。
- ③ 节点指针数组的使用避免了动态内存分配，有利于提升访问速度，并且更易于编程实现。





1.主程序流程：首先进入压缩软件的开始界面，显示提示信息。用户通过键盘输入数字1进行压缩文件，输入2则进行解压缩，若输入错误会提示用户重新输入。当压缩文件时，用户输入含有后缀名的源文件，并输入压缩后的.huff文件的名称，若文件打开失败会提示错误信息。当源文件正常打开后程序会读取文件中的字符并建立字符集频率表，接下来程序根据建立的字符集频率表构建哈夫曼树并对源文件进行哈夫曼编码，最后将字符种类数、编码表和压缩数据写入.huff文件完成压缩。当解压文件时，用户输入.huff文件并输入含有后缀名的解压缩后的文件名称，若文件打开失败会提示错误信息。当.huff文件正常打开后，程序会重建解码表并由此将.huff文件中的编码还原为源文件中的字符，实现解压缩操作

```
// ===== 主函数 =====  
int main() {  
    int choice;  
    string inputFile, outputFile;  
  
    cout << "=== 哈夫曼文件压缩工具 ===" << endl;  
    cout << "1. 压缩文件\n2. 解压文件\n请选择操作: ";  
    cin >> choice;  
    while (choice != 1 && choice != 2) {  
        cout << "输入错误! 请重新输入" << endl;  
        cin >> choice;  
    }  
    cin.ignore(); // 清除输入缓冲区的换行符  
  
    cout << "输入文件名: ";  
    getline(cin, inputFile);  
    cout << "输出文件名: ";  
    getline(cin, outputFile); // 文件名需要加后缀  
  
    if (choice == 1) {  
        // 压缩流程  
        map<char, int> freqMap = FileManager::countFrequency(inputFile);  
        HuffmanEncoder encoder;  
        encoder.buildTree(freqMap);  
        FileManager::writeCompressed(inputFile, outputFile, encoder.getCodeTable());  
        cout << "压缩完成! 压缩文件已保存为 " << outputFile << endl;  
    } else if (choice == 2) {  
        // 解压流程  
        FileManager::decompress(inputFile, outputFile);  
        cout << "解压完成! 文件已保存为 " << outputFile << endl;  
    } else {  
        cerr << "错误: 无效的选择!" << endl;  
    }  
  
    return 0;  
}
```

2. 统计字符频率：
通过countFrequency函数实现，这里使用map容器实现字符到出现频率的映射。打开源文件后，程序逐一读取字符，并将该字符所对应的频率加一，读取完毕后关闭文件。

```
// 统计字符频率 (返回频率表)
static map<char, int> countFrequency(const string& filename) {
    ifstream fin(filename, ios::binary);
    if (!fin) {
        cerr << "错误: 无法打开文件 " << filename << endl;
        exit(1);
    }

    map<char, int> freqMap;
    char ch;
    while (fin.get(ch)) { // 逐个字符读取
        freqMap[ch]++;    // 统计频率
    }
    fin.close();
    return freqMap;
}
```

3. 字符哈夫曼编码生成：
通过buildCode函数实现。该算法是在哈夫曼树构建完成后，采取从根节点开始的深度优先遍历，使用递归的方法实现。当根节点为空时结束递归，当根节点不为空时，对于左子树路径编码追加“0”后向下递归，对于右子树路径编码追加“1”后向下递归。当到达叶子结点时，将字符和对应的路径编码存入编码表。

```
// 递归生成字符编码 (核心方法)
void buildCode(HuffmanNode* node, string code) {
    if (!node) return; // 递归终止条件

    // 叶子节点存储编码
    if (node->isLeaf()) {
        codeTable[node->data] = code;
        return;
    }

    // 左子树编码追加0, 右子树追加1
    buildCode(node->left, code + "0");
    buildCode(node->right, code + "1");
}
```

4.构建哈夫曼树：首先根据建立的字符集频率表，为每个字符创建叶子结点，形成只有根结点的二叉树，令其权值为出现频率，得到森林。在森林中选取两棵根结点权值最小的树分别作为左右子树，构造一棵新的二叉树，置新二叉树根结点权值为其左右子树根结点权值之和。然后在森林中删除这两棵树，同时将新得到的二叉树加入森林中。重复上述步骤，直到森林中只剩一棵树为止，这棵树即为所构建的哈夫曼树。

```
// 构建哈夫曼树（核心方法）
void buildTree(const map<char, int>& freqMap) {
    const int MAX_NODES = 512; // 最大节点数（256个字符需要511个节点）
    HuffmanNode* nodes[MAX_NODES] = {nullptr}; // 初始化指针数组
    int nodeCount = 0;

    // 处理空文件特殊情况
    if (freqMap.empty()) {
        root = nullptr;
        codeTable.clear();
        return;
    }

    // 第一步：创建所有叶节点
    for (map<char, int>::const_iterator it = freqMap.begin();
         it != freqMap.end(); ++it) {
        nodes[nodeCount++] = new HuffmanNode(it->first, it->second);
    }

    // 第二步：构建哈夫曼树
    while (nodeCount > 1) {
        // 按频率升序排序
        sortNodes(nodes, nodeCount);

        // 取出两个最小节点
        HuffmanNode* left = nodes[0];
        HuffmanNode* right = nodes[1];

        // 创建新父节点（字符设为'\0'）
        HuffmanNode* parent = new HuffmanNode('\0', left->freq + right->freq);
        parent->left = left;
        parent->right = right;

        // 更新节点数组
        nodes[0] = parent; // 父节点占据第一个位置
        for (int i = 1; i < nodeCount-1; ++i) {
            nodes[i] = nodes[i+1]; // 后续节点前移
        }
        nodeCount--; // 总节点数减1
    }

    root = nodes[0]; // 最后的节点即为根节点
    buildCode(root, ""); // 生成编码表
}
```

5. 写入压缩文件：通过writeCompressed函数实现。首先以二进制模式打开源文件，若为空文件自动返回。接下来将字符种类数和编码表写入头文件，方便以后得解压缩操作。然后逐个读取源文件中的字符信息并查找其哈夫曼编码，在存储时使用位缓冲机制：首先建立8位缓冲区，然后遍历编码的每个位并左移缓冲区将当前位添加进来。当缓冲区满八位时就写入文件并重置缓冲区，不满八位时补零后写入文件，压缩完成后关闭源文件。

```
// 写入压缩文件（核心方法）
static void writeCompressed(const string& inputFile,
                           const string& outputFile,
                           const map<char, string>& codeTable) {
    ifstream fin(inputFile, ios::binary);
    ofstream fout(outputFile, ios::binary);

    // 写入字符种类数（4字节）
    int charCount = codeTable.size();
    fout.write(reinterpret_cast<const char*>(&charCount), sizeof(int));

    // 空文件特殊处理
    if (charCount == 0) {
        fin.close();
        fout.close();
        return;
    }

    // 写入编码表（字符+编码长度+编码内容）
    for (map<char, string>::const_iterator it = codeTable.begin();
         it != codeTable.end(); ++it) {
        fout.put(it->first); // 写入字符（1字节）
        unsigned char len = it->second.length();
        fout.put(len); // 编码长度（1字节）
        fout.write(it->second.c_str(), len); // 编码内容
    }

    // 压缩数据
    unsigned char buffer = 0; // 位缓冲区
    int bitCount = 0; // 当前缓冲位数
    char ch;

    while (fin.get(ch)) {
        const string& code = codeTable.at(ch);
        for (size_t i = 0; i < code.size(); ++i) {
            buffer = (buffer << 1) | (code[i] - '0'); // 左移并添加位
            if (++bitCount == 8) { // 缓冲区满
                fout.put(buffer);
                buffer = bitCount = 0;
            }
        }
    }

    // 处理剩余位（不足8位时补0）
    if (bitCount > 0) {
        buffer <<= (8 - bitCount); // 左移补零
        fout.put(buffer);
    }

    fin.close();
    fout.close();
}
```

6.实现文件压缩的原理：通过对字符使用0和1码元进行哈夫曼编码，使出现概率大的字符对应短码，出现概率小的字符对应长码。在完成编码生成压缩文件时，将每个字符对应的二进制哈夫曼编码按照八位一个字节写入文件，这样得到的压缩文件字节数比源文件要少。

7. 解压文件：通过decompress函数实现。首先以二进制模式打开.huff文件，若为空文件自动返回。接下来读取文件头的内容，重建解码表，得到编码字符串到原字符的映射。在解压缩时，逐字节读取压缩数据，根据哈夫曼编码是前缀码的特性，进行前缀匹配解码，匹配成功则写入字符，若匹配失败则重新读取压缩数据后再进行匹配。重复以上操作，直至完成解压缩为止。

```
// 解压文件 (核心方法)
static void decompress(const string& inputFile, const string& outputFile) {
    ifstream fin(inputFile, ios::binary);
    ofstream fout(outputFile, ios::binary);

    // 读取字符种类数
    int charCount;
    fin.read(reinterpret_cast<char*>(&charCount), sizeof(int));

    // 空文件处理
    if (charCount == 0) {
        fin.close();
        fout.close();
        return;
    }

    // 重建解码表 (编码到字符的映射)
    map<string, char> decodeMap;
    for (int i = 0; i < charCount; ++i) {
        char data;
        unsigned char codeLen;
        fin.get(data);
        fin.read(reinterpret_cast<char*>(&codeLen), 1);

        string code;
        code.resize(codeLen);
        fin.read(&code[0], codeLen); // 读取编码内容

        decodeMap[code] = data; // 建立映射关系
    }

    // 解码数据
    string bitStr; // 累积的二进制位字符串
    char byte;
    while (fin.get(byte)) {
        // 将字节转为8位二进制字符串
        for (int i = 7; i >= 0; --i) {
            bitStr += (byte & (1 << i)) ? '1' : '0';
        }

        // 持续匹配编码
        bool found;
        do {
            found = false;
            for (map<string, char>::const_iterator it = decodeMap.begin();
                it != decodeMap.end(); ++it) {
                if (bitStr.substr(0, it->first.size()) == it->first) {
                    fout.put(it->second); // 写入解码字符
                    bitStr = bitStr.substr(it->first.size()); // 移除已解码部分
                    found = true;
                    break;
                }
            }
        } while (!found && !bitStr.empty());

        bitStr = bitStr.substr(0, bitStr.size() - 8);
    }

    fin.close();
    fout.close();
}
```



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

3

测试与结论

打开程序界面

```
=== 哈夫曼文件压缩工具 ===  
1. 压缩文件  
2. 解压文件  
请选择操作: |
```

压缩文件，压缩后数据存储在1组-灰度图.huff

```
=== 哈夫曼文件压缩工具 ===  
1. 压缩文件  
2. 解压文件  
请选择操作: 1  
输入文件名: 1组-灰度图.bmp  
输出文件名: 1组-灰度图.huff  
压缩完成! 压缩文件已保存为 1组-灰度图.huff  
  
D:\lessons_and_papers\computer_lessons\C_plus_plus\Codes\codes\x64\Debug\codes.exe (进程 27996)已退出, 代码为 0 (0x0).  
按任意键关闭此窗口. . .
```



对1组-灰度图.huff进行解压缩，得到2组-灰度图.bmp

```
=== 哈夫曼文件压缩工具 ===  
1. 压缩文件  
2. 解压文件  
请选择操作: 2  
输入文件名: 1组-灰度图.huff  
输出文件名: 2组-灰度图.bmp  
解压完成! 文件已保存为 2组-灰度图.bmp  
  
D:\lessons_and_papers\computer_lessons\C_plus_plus\Codes\codes\x64\Debug\codes.exe (进程 28816)已退出, 代码为 0 (0x0)。  
按任意键关闭此窗口. . .|
```

源文件大小为8254字节，压缩后变为6844字节，压缩率1.206，说明我们设计的压缩软件可以有效压缩文件。
源文件和解压文件的对比如下图：

第1组



第1组



按照同样的步骤对英文文本“novel1.txt”进行压缩，源文件大小为523110字节，压缩后的“novel1.huff”文件变为275269字节，压缩比为1.900，说明我们设计的压缩软件可以有效压缩英文文本。源文件和解压文件的部分文本内容对比如下图：

He was an old man who fished alone in a skiff in the Gulf Stream and he had gone eighty-four days now without taking a fish. In the first forty days a boy had been with him. But after forty days without a fish the boy's parents had told him that the old man was now definitely and finally salao, which is the worst form of unlucky, and the boy had gone at their orders in another boat which caught three good fish the first week. It made the boy sad to see the old man come in each day with his skiff empty and he always went down to help him carry either the coiled lines or the gaff and harpoon and the sail that was furled around the mast. The sail was patched with flour sacks and, furled, it looked like the flag of permanent defeat.

He was an old man who fished alone in a skiff in the Gulf Stream and he had gone eighty-four days now without taking a fish. In the first forty days a boy had been with him. But after forty days without a fish the boy's parents had told him that the old man was now definitely and finally salao, which is the worst form of unlucky, and the boy had gone at their orders in another boat which caught three good fish the first week. It made the boy sad to see the old man come in each day with his skiff empty and he always went down to help him carry either the coiled lines or the gaff and harpoon and the sail that was furled around the mast. The sail was patched with flour sacks and, furled, it looked like the flag of permanent defeat.

通过以上的对比，源文件在经过压缩解压后不会发生失真，符合压缩软件的设计要求。



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

4 总结与思考

题目难点要点

1.哈夫曼树的构建：读取文件得到字符集频率表并建立哈夫曼树，由此得到哈夫曼编码

2.文件的读取和写入：压缩文件头结构的设计，通过合理的编码存储方式实现文件压缩

3.内存资源的管理：树节点递归与释放，二叉链表中指针的精确管理，防止内存泄漏。

本组工作特点

1.哈夫曼编码压缩文件的实现：实现了完整的哈夫曼压缩解压流程，包括频率统计、树构建、编码生成、压缩和解压。

2.Map容器的应用：本程序采用map容器建立了编码表、频率表和解码表,构建了各种映射关系，便于数据的快速读取、查找和修改等操作。

3.文件格式设计及其读写操作：通过在压缩文件中添加头文件部分，使得解压时无需依赖外部存储的编码表等额外信息。采用位缓冲机制处理变长编码写入，提高压缩效率。

本组改进方向

1.对源文件字符种类个数有限制：本程序最多只能处理含有256个字符的文件，对于字符个数较多的大文件无法正常压缩，可以进一步改善使其支持更多字符种类。

2.压缩率进一步优化：支持动态哈夫曼编码，并增加LZ77预处理。

3.解压缩性能优化：解压缩使用前缀匹配法效率较低，可以考虑重构哈夫曼树进行解码



感谢观看

Thanks for Your Attention